Smart Campus Resource Booking System

Programming 512 - Case Study Documentation (100 marks)
Richfield Graduate Institution

1. Purpose

The institution shares computer labs, multimedia projectors, seminar rooms and presentation equipment across several campuses. Manual bookings caused double bookings, poor accountability and no central reports. This Python desktop application provides one consistent system with a Tkinter graphical interface, role-based access, campus-specific booking rules and SQLite storage.

2. User roles

Role	Can do	Campus access
Lecturer	Register, log in, select a campus, view resource availability, book, view active bookings, cancel future bookings, view history	Any campus
Campus Administrator	Add, update and remove resources; view all bookings; check availability; campus report and usage trends	Own campus only
System Operator	Everything an administrator can do plus all bookings and resources across campuses and the cross-campus comparison	All campuses

3. Campus booking policies

Rule	Johannesburg	Pretoria	Durban
Max duration per booking	120 min	180 min	90 min
Booking hours	08:00-18:00	07:00-20:00	09:00-17:00
Max bookings per lecturer per day	3	4	2
Book up to (days ahead)	30	60	14
Extra rule	None	Projectors: 60 min max	Seminar rooms (priority access): 1 day notice

Each campus has a BookingRule object. The base class BookingRule holds the shared checks; PretoriaRule and DurbanRule override check_booking() to add their own rule (inheritance and polymorphism). The rules are in campus.py, separate from the core logic, so a new campus is added with one line in the campuses dictionary.

4. Program structure

File	Responsibility
main.py	Starts the application
gui.py	Tkinter screens: LoginScreen, RegisterScreen, CampusScreen, MainScreen and the pages for booking, my bookings, managing resources and lists/reports
booking_manager.py	Registration, login, resource management, availability, booking, overlap check, cancellation, validation
campus.py	Campus class, BookingRule classes, campuses dictionary
users.py	User base class and Lecturer, CampusAdministrator, SystemOperator classes with permission sets
database.py	All SQLite code, table creation and starting data
reports.py	Campus report, cross-campus comparison and text formatting for the lists

File	Responsibility
test_system.py	32 automatic tests of the program logic

5. Database design (SQLite)

Table	Columns
campuses	name (PK), location
users	username (PK), full_name, password_hash, role, campus (FK)
resources	resource_id (PK, unique identifier), name, resource_type, campus (FK), status
bookings	booking_id (PK), resource_id, resource_name, resource_type, username (FK), campus (FK), booking_date, start_time, duration_minutes, status, created_at

All statements use ? placeholders. Passwords are stored as SHA-256 hashes. Cancelled bookings are kept with status Cancelled so the history stays complete.

6. How the main requirements are met

Requirement	Implementation
Lists, dictionaries, sets	campuses dictionary; rows as dictionaries; usage counters; permission sets; lists of resources and bookings
OOP for roles	User with three child classes; private attributes with get methods
Unique resource ID, reject duplicates	resource_id is the primary key; add_resource() checks first and also catches IntegrityError
Prevent overlapping bookings	overlaps(): two slots clash when each starts before the other ends; cancelled bookings do not block
Booking linked to lecturer and campus	bookings.username and bookings.campus store both
Real-time availability	Resource list recalculated on every keystroke and every 10 seconds; green/red/grey
Cancel future bookings	cancel_booking() only for the owner and only before the start time; saved to the database
Reports	Total bookings, most used resources, average duration, hours per resource, weekday trend, cross-campus table and observations
Validation and error handling	Date/time/duration checks, permission checks, try/except around database calls, message boxes

7. Testing

The 32 tests in test_system.py use a temporary database and check: registration and login, passwords not stored in plain text, role permissions, every campus rule (including the exact limits), overlap and adjacent bookings, past dates, bad input, unavailable resources, cancellation rules, resource add/update/remove with duplicate and access-control checks, report figures, persistence between runs and adding a new campus.

```
Run them with: python test_system.py
```

8. Running the program

Requires Python 3.8 or newer (tkinter and sqlite3 are included). In PyCharm open main.py and choose Run, or use: python main.py. The database file is created automatically on the first run.

Role	Username	Password
Lecturer	lecturer1 / lecturer2	lecturer123

Role	Username	Password
Campus Administrator	admin_jhb / admin_pta / admin_dbn	admin123
System Operator	operator	operator123

GitHub repository: https://github.com/YOUR-USERNAME/smart-campus-booking-system (replace YOUR-USERNAME with your GitHub username - see README.md).